

SAE5 ROM.03 : Déployer et gérer les services ROM

Formation par apprentissage

RT3-FA-A1

23 janvier 2026

Créé par : Indira LUIS / Noa ETOURNEAU

SOMMAIRE

Table des matières

PRESENTATION DU PROJET.....	2
ÉTAPE 1 – DEPLOIEMENT D’UN SERVEUR ASTERISK SOUS ALPINE LINUX.....	4
CREATION DE LA MACHINE VIRTUELLE ALPINE LINUX	4
INSTALLATION DU SERVEUR ASTERISK	5
VERIFICATION DU FONCTIONNEMENT DU SERVEUR ASTERISK.....	5
SAUVEGARDE DES FICHIERS DE CONFIGURATION.....	6
ÉTAPE 2 - CONNECTER DEUX TELEPHONES IP (FANVIL ET CISCO) SUR LE SERVEUR TESTER LA COMMUNICATION EN MODE VOIX	7
CREATION DES COMPTES SIP (PJSIP.CONF) SUR LE SERVEUR ASTERISK	7
CREATION DU FICHIER EXTENSIONS.CONF.....	8
REINITIALISATION ET CONFIGURATION DU TELEPHONE FANVIL	9
ENREGISTREMENT DU TELEPHONE CISCO VIA SERVEUR TFTP.....	10
ILLUSTRATION DES DEUX METHODES D’ENREGISTREMENT DES TERMINAUX	13
MISE EN PLACE DES BOITES VOCALES POUR LES DEUX COMPTES	15
REALISATION D’UN ECHANGE VOIX ENTRE LES DEUX TERMINAUX	16
ÉTAPE 3 - ACCEDER A L’ADMINISTRATION D’ASTERISK VIA SON SERVEUR HTTP	17
ACTIVATION DU SERVEUR HTTP INTEGRE.....	17
DEVELOPPEMENT D’UNE INTERFACE WEB SIMPLE	18
ÉTAPE 4 - DEPLOYER UN SERVEUR WEB SUR UNE MACHINE VIRTUELLE AVEC LA DISTRIBUTION ALPINE	21
INSTALLATION ET CONFIGURATION DU SERVEUR WEB.....	21
ÉTAPE 5 - INTEGRER UNE PAGE WEB SUR LE SERVEUR PRECEDENT POUR REALISER UN ECHANGE VOIX A L’AIDE DE LA LIBRAIRIE SIP.JS	23
QU’EST-CE QUE WEBRTC ?.....	23
PRINCIPE DE MISE EN PLACE POUR LA PAGE WEB AVEC SIP.JS.....	24
CONFIGURATION DU SERVEUR NGINX (WEB + WEBSOCKET).....	25
CONFIGURATION COTE ASTERISK (WEBRTC)	32
TESTS ET VALIDATION DE LA SOLUTION WEBRTC / SIP	34
ÉTAPE 6 – COMPLETER L’INTERFACE PRECEDENTE POUR INTEGRER UNE MESSAGERIE INSTANTANEE	37
CHOIX TECHNIQUE : ALTERNATIVE AU PROTOCOLE SIP	37
PREPARATION DE L’ENVIRONNEMENT PHP SUR ALPINE LINUX	37
CREATION DE L’API DE MESSAGERIE (CHAT.PHP)	38
MISE EN PLACE DU STOCKAGE DES MESSAGES.....	39
TESTS ET VALIDATION DE LA MESSAGERIE INSTANTANEE :	40
LES PROBLEMES RENCONTRES	42
CONCLUSION.....	43

Présentation du projet

Quel est le contexte et les enjeux ?

Dans le cadre de cette SAE, notre objectif est de mettre en œuvre des services de téléphonie et de communication multimédia en nous appuyant sur la solution open-source **Asterisk**. Ce logiciel constitue un IPBX complet permettant de fournir des services de voix sur IP, de visioconférence et de messagerie instantanée, utilisés aussi bien dans les réseaux d'entreprises que dans les infrastructures des opérateurs.

Le projet s'inscrit dans le contexte des métiers du **Réseaux & Télécommunications – parcours Réseaux Opérateurs et Multimédia (ROM)**, où le professionnel est amené à déployer, administrer et sécuriser des services tels que l'accès Internet, la téléphonie IP et les services vidéo. Il répond également aux enjeux actuels liés à la **virtualisation**, à l'**automatisation** et à l'**intégration de services multimédias via des interfaces Web**.

Tout au long de ce projet, nous nous appuyons sur la documentation officielle d'Asterisk ainsi que sur les ressources associées aux bibliothèques Web utilisées. L'objectif est de comprendre le fonctionnement d'un serveur de téléphonie IP, de le configurer, puis de l'interfacer avec une application Web permettant des communications voix et vidéo en temps réel.

Quelles sont les technologies utilisées ? :

Le projet repose sur les éléments suivants :

- **Asterisk** : serveur de téléphonie IP open-source permettant la gestion des appels, des comptes SIP et des services multimédias.
- **Alpine Linux** : distribution Linux légère utilisée pour le déploiement des machines virtuelles, favorisant la performance et la sécurité.
- **sip.js** : bibliothèque JavaScript permettant d'établir des communications SIP directement depuis un navigateur Web.
- **Serveur Web** (Apache ou Nginx) : utilisé pour héberger l'interface Web de communication.
- **Téléphones IP** : équipements CISCO et FANVIL utilisés pour tester la communication voix.
- **Machines virtuelles** : chaque service (Asterisk, serveur Web) est déployé sur une machine virtuelle dédiée afin de reproduire une architecture réaliste et modulaire.

Comment est organisé le projet ? :

Le projet s'est déroulé sur plusieurs séances de travaux pratiques, pour un volume total estimé à **21 heures**, en combinant TP encadrés et non encadrés. L'organisation du projet a suivi une progression par étapes, chaque phase devant être validée avant de passer à la suivante.

- **Étapes 1 à 3** : déploiement et configuration du serveur Asterisk, connexion des téléphones IP et accès à l'administration via le serveur HTTP.
- **Étapes 4 et 5** : déploiement d'un serveur Web et intégration d'une interface Web permettant les communications voix à l'aide de sip.js.
- **Étapes 6 à 8** : extension de l'interface Web pour intégrer la vidéo, la messagerie instantanée et la réalisation d'un portail de visioconférence.
- **Étape 9** : audit de sécurité des échanges entre le portail Web et le serveur Asterisk.

Type	Sem.	Filière	Groupe	Enseignant	Lieu	Début	Fin	Durée (h)
TP	S5	FA	Groupe 1		H0-05	17/09 09:00	17/09 12:00	3.0
TP	S5	FA	Groupe 1		H0-05	22/09 13:30	22/09 16:30	3.0
TP	S5	FA	Groupe 1		H0-05	16/10 13:00	16/10 16:00	3.0
TP	S5	FA	Groupe 1		H0-05	17/11 13:30	17/11 16:30	3.0
TP	S5	FA	Groupe 1		H0-05	19/11 13:30	19/11 16:30	3.0
TP	S5	FA	Groupe 1		H0-05	09/12 13:30	09/12 16:30	3.0
TP	S5	FA	Groupe 1		H0-05	10/12 13:30	10/12 16:30	3.0

Total d'heures : 21.0

Tableau récapitulatif de l'architecture :

Machine virtuelle	Adresse IP	Masque	Type de serveur / OS	Rôle principal
Asterisk-Alpine	10.16.20.94	/12	Alpine Linux	Serveur de téléphonie IP (Asterisk, SIP, RTP, WebRTC)
Alpine-std	10.16.20.96	/12	Alpine Linux	Serveur TFTP (provisionnement des téléphones)
Alpine-std	10.16.21.6	/12	Alpine Linux	Serveur Web (NGINX, SIP.js, WebRTC)
Ubuntu-desktop	10.16.21.7	/12	Ubuntu Desktop	Poste client de test WebRTC (PC 1)
Ubuntu-desktop	10.16.21.8	/12	Ubuntu Desktop	Poste client de test WebRTC (PC 2)

Étape 1 – Déploiement d'un serveur Asterisk sous Alpine Linux

Cette première étape a pour objectif de déployer une machine virtuelle sous **Alpine Linux**, d'y installer le serveur de téléphonie **Asterisk**, puis de vérifier le bon fonctionnement du service. Cette étape constitue la base de l'ensemble du projet, car toutes les communications voix, vidéo et messagerie reposeront sur ce serveur.

Création de la machine virtuelle Alpine Linux

La machine virtuelle dédiée au serveur Asterisk a été créée à partir de la distribution **Alpine Linux**, choisie pour sa légèreté et sa rapidité de déploiement.

Pour la **machine Virtuelle Alpine** il faut mettre en login **root** et en mot de passe **progrtr00**.

Lors de la création de cette machine virtuelle, les paramètres suivants ont été appliqués :

Carte réseau	Mode réseau	Interface physique associée	VLAN
1 interface réseau	Bridge	eth1	800

Cette configuration permet à la machine virtuelle d'accéder au réseau de l'IUT et de communiquer avec les autres équipements du projet.

L'adresse IP attribuée au serveur Asterisk-Alpine est la suivante : **10.16.20.94**

```
rt-mo:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:7a:8b:81 brd ff:ff:ff:ff:ff:ff
    inet 10.16.20.94/12 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe7a:8b81/64 scope link
        valid_lft forever preferred_lft forever
```

Installation du serveur Asterisk

Une fois la machine virtuelle Alpine opérationnelle, l'installation du serveur Asterisk a été réalisée à l'aide du gestionnaire de paquets **apk**.

Avant d'installer les paquets, il est nécessaire de configurer les proxies afin d'accéder aux dépôts :

```
export http_proxy=http://cache-etu.univ-artois.fr:3128  
export https_proxy=http://cache-etu.univ-artois.fr:3128
```

Ensuite, une mise à jour de la liste des paquets et l'installation d'Asterisk se fait avec les commandes suivantes :

```
apk update  
apk add asterisk
```

Afin de faciliter l'administration du système, des paquets complémentaires ont également été installés :

```
apk add nano  
apk add openssh
```

Ces outils permettent respectivement l'édition de fichiers de configuration et l'accès distant sécurisé au serveur.

Vérification du fonctionnement du serveur Asterisk

Après l'installation, il est nécessaire de vérifier que le service Asterisk est bien opérationnel sur la machine virtuelle.

La commande suivante permet de vérifier l'état du service :

```
service asterisk status
```

Si le service n'est pas en cours d'exécution, il peut être démarré manuellement avec la commande :

```
service asterisk start
```

Pour redémarrer le service Asterisk, la commande suivante est utilisée :

```
service asterisk restart
```

Le bon fonctionnement du serveur peut également être vérifié en accédant à la console Asterisk à l'aide de la commande :

```
asterisk -rvvv
```

Cette commande permet d'accéder à l'interface en ligne de commande d'Asterisk et de visualiser les messages du serveur en temps réel.

```
rt-mv:/etc/asterisk# service asterisk restart
* Stopping asterisk PBX now ...
* Starting asterisk PBX (as asterisk) ...
rt-mv:/etc/asterisk# service asterisk status
* status: started
```

Sauvegarde des fichiers de configuration

Avant toute modification des fichiers de configuration, une sauvegarde du fichier original **pjsip.conf** a été réalisée afin de pouvoir revenir à une configuration fonctionnelle en cas d'erreur.

Les commandes suivantes ont été utilisées :

```
cd /etc/asterisk/
cp pjsip.conf pjsip.conf.src
```

Le fichier **pjsip.conf** contient la déclaration des comptes SIP et sera utilisé dans les étapes suivantes pour configurer les téléphones IP et les utilisateurs Web.

Étape 2 - Connecter deux téléphones IP (Fanvil et Cisco) sur le serveur tester la communication en mode voix

Cette étape a pour objectif de connecter deux téléphones IP de marques différentes (**FANVIL** et **CISCO**) au serveur Asterisk, de mettre en place les comptes SIP associés, puis de tester la communication en mode voix. Elle permet également d'illustrer deux méthodes d'enregistrement des terminaux SIP et de valider les échanges à l'aide d'une analyse réseau.

Création des comptes SIP (pjsip.conf) sur le serveur Asterisk

Afin d'éviter toute erreur liée à une configuration existante, le fichier de configuration **pjsip.conf** est recréé à partir d'un fichier vierge.

```
cd /etc/asterisk/  
nano pjsip.conf
```

Deux comptes SIP sont créés pour les téléphones IP, selon la convention suivante :

- **FanvilXX**
- **CiscoXX**

Où **XX** correspond au numéro du poste. Dans notre cas, nous travaillons sur le **poste 12**, ce qui conduit à la création des comptes **Fanvil12** et **Cisco12**.

Les paramètres principaux sont les suivants :

Compte SIP	Mot de passe	Context	Numéro interne	Adresse IP
Fanvil12	1234	internal	1201	10.17.1.12
Cisco12	5678	internal	1202	10.17.1.32

Pour le Fanvil12 :

```
=====Fanvil12  
  
[Fanvil12]  
type = endpoint  
context = internal  
auth = Fanvil12-auth  
aors = Fanvil12  
transport = transport-udp  
disallow = all  
allow = ulaw  
  
[Fanvil12-auth]  
type = auth  
auth_type = userpass  
password = 1234  
username = Fanvil12  
  
[Fanvil12]  
type = aor  
max_contacts = 1
```


Pour le Cisco12 :

```
====Cisco12

[Cisco12]
type = endpoint
context = internal
auth = Cisco12-auth
aors = Cisco12
transport = transport-udp
disallow = all
allow = ulaw

[Cisco12-auth]
type = auth
auth_type = userpass
password = 5678
username = Cisco12

[Cisco12]
type = aor
max_contacts = 1
```

Création du fichier `extensions.conf`

Le fichier **`extensions.conf`** joue un rôle central dans le fonctionnement d'Asterisk. Il permet de définir le **plan de numérotation**, c'est-à-dire les règles qui indiquent au serveur comment traiter les appels entrants : vers quel poste les diriger, quelles actions effectuer et dans quel contexte appliquer ces règles.

Sans ce fichier, Asterisk ne sait pas comment acheminer les appels entre les différents terminaux SIP. Il est donc indispensable pour permettre la communication voix entre les téléphones FANVIL et CISCO. L'édition du fichier de configuration se fait à l'aide de la commande suivante :

```
cd /etc/asterisk/
nano extensions.conf
```

```
GNU nano 5.9                                extensions.conf
[internal]
exten => 1201,1,Dial(PJSIP/Fanvil12)
exten => 1202,1,Dial(PJSIP/Cisco12)
```

Dans ce fichier, un contexte nommé **`internal`** est défini. Ce contexte correspond à celui renseigné dans le fichier `pjsip.conf` pour les comptes SIP. Il contient les règles permettant d'associer un numéro interne à chaque téléphone. Lorsque l'un des téléphones compose le numéro interne de l'autre, Asterisk consulte le fichier `extensions.conf`, identifie la règle correspondante et établit l'appel entre les deux terminaux.

Une fois les comptes configurés, le service Asterisk est redémarré afin de prendre en compte les modifications :

```
service asterisk restart
```

Réinitialisation et configuration du téléphone FANVIL

Avant l'enregistrement du téléphone FANVIL sur le serveur Asterisk, une réinitialisation matérielle est effectuée afin de repartir d'une configuration propre.

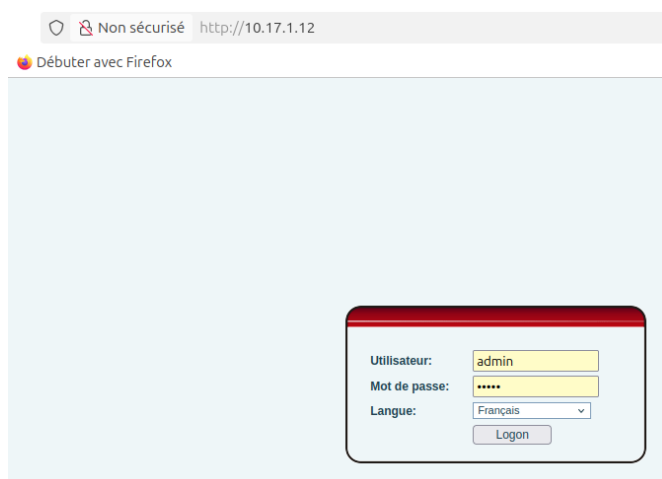
Procédure de réinitialisation du téléphone FANVIL :

- Débrancher le câble RJ45
- Rebrancher tout en appuyant sur la touche “#”
- Attendre l'apparition du message Post Mode
- Mettre le code *#168
- Attendre l'apparition du message Phone reset
- Déconnecter et reconnecter la fiche RJ45 sur le port du téléphone.

Une fois le téléphone redémarré, l'interface Web du FANVIL est accessible via son adresse IP (dans notre cas **10.17.1.12**) depuis un navigateur.

Identifiants de connexion :

- Utilisateur : **admin**
- Mot de passe : **admin**



Dans le menu **Line > SIP**, les paramètres SIP correspondant au compte **Fanvil12** sont renseignés (serveur SIP, identifiant, mot de passe).

Paramètres de base >>	
État de la ligne	Enregistré
Nom d'utilisateur	Fanvil12
Affichage du nom	Fanvil12
Realm	
Activer	☒
Nom d'authentification	Fanvil12
Mot de passe d'authentification	****
Server Name	10.16.20.94
SIP Server 1	
Adresse du serveur Proxy SIP	10.16.20.94
Port serveur Proxy SIP	5060
Protocole de transport	UDP
Expiration de l'enregistrement	3600 Seconde
SIP Server 2	
Adresse du serveur Proxy SIP	
Port serveur Proxy SIP	5060
Protocole de transport	UDP
Expiration de l'enregistrement	3600 Seconde

Nous avons également changé l’heure et la date pour mettre le serveur à jour :

Paramètres du serveur de temps réseau

Heure synchronisé via SNTP

☐

?

Heure synchronisé via DHCP

☐

?

Serveur de temps primaire

?

Serveur de temps secondaire

?

Fuseau horaire

(UTC+1) Albania,Austria,Belgium,Caici

Fréquence de re-synchro

Seconde

?

Format date

Horloge au format 12 heures

☐

Format date

DD MMM WW

Paramètres heure d'été

Location

France(Paris)

Enregistrement du téléphone CISCO via serveur TFTP

Contrairement au téléphone FANVIL, le téléphone **CISCO** nécessite l’utilisation d’un serveur **TFTP (Trivial File Transfer Protocol)** afin de récupérer automatiquement son fichier de configuration. Le serveur TFTP contient tous les fichiers nécessaires au paramétrage du téléphone, car l’interface Web du CISCO ne permet pas de modifier directement les réglages.

Lancer le script **MachinesVirtuelles** dans un terminal de la machine hôte. Pour la machine serveur TFTP **Alpine-std** il faut mettre en login **root** et en mot de passe **progrtr00**.

Lors de la création de cette machine virtuelle, les paramètres suivants ont été appliqués :

Carte réseau	Mode réseau	Interface associée	Adresse IP	Adresse MAC
1 interface réseau	Bridge	eth1	10.16.20.96	08:00:27:a4:8e:c6

```
rt-mv:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
   link/ether 08:00:27:a4:8e:c6 brd ff:ff:ff:ff:ff:ff
   inet 10.16.20.96/12 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fea4:8ec6/64 scope link
       valid_lft forever preferred_lft forever
```

Après avoir lancé la machine virtuelle TFTP, nous devons installer les paquets suivants :

```
apk add tftp-hpa
apk add openssh
apk add nano
apk add unzip
```

Ensuite, il faut démarrer les services avec :

```
service in.tftpd start
service sshd start
```

Nous allons maintenant nous occuper des fichiers de configuration TFTP. Pour cela, il faudra se placer dans le répertoire du serveur TFTP et télécharger l'archive contenant les fichiers de configuration :

```
cd /var/tftpboot/
wget http://iut-rt.univ-artois.fr/tph/Archive.zip
```

Décompresser l'archive :

```
unzip Archive.zip
```

Les fichiers suivants sont ajoutés : **DialTemplate.xml** et **SEP_MAC-CISCO_.cnf.xml**

```
rt-mv:/var/tftpboot# unzip Archive.zip
Archive: Archive.zip
  inflating: DialTemplate.xml
  inflating: SEP_MAC-CISCO_.cnf.xml
```

Il faudra s'occuper de la configuration spécifique de notre téléphone CISCO. Premièrement, renommer le fichier de configuration en remplaçant **_MAC-CISCO_** par l'adresse MAC du téléphone sans les deux points.

Cela donna : **SEP544A00369ECB.cnf.xml**

```
rt-mv:/var/tftpboot# cp SEP_MAC-CISCO_.cnf.xml SEP080027A48EC6.cnf.xml
rt-mv:/var/tftpboot# ls
Archive.zip                SEP080027A48EC6.cnf.xml
DialTemplate.xml           SEP_MAC-CISCO_.cnf.xml
rt-mv:/var/tftpboot#
```

Ensuite, modifier le fichier avec *nano* pour renseigner : **nano SEP544A00369ECB.cnf.xml**

- L'adresse IP du serveur TFTP :

```
<securedSipPort>5061</securedSipPort>
</ports>
<processNodeName>10.16.20.96</processNodeName>
</callManager>
</member>
```

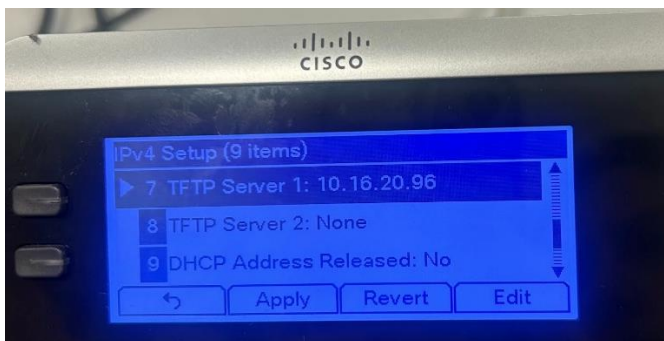
- Les informations du compte SIP correspondant au poste :

```
<phoneLabel>Cisco12</phoneLabel>
<featureLabel>Cisco12</featureLabel>
<authName>Cisco12</authName>
<contact>Cisco12</contact>
```

Enfin, relancer le service TFTP :

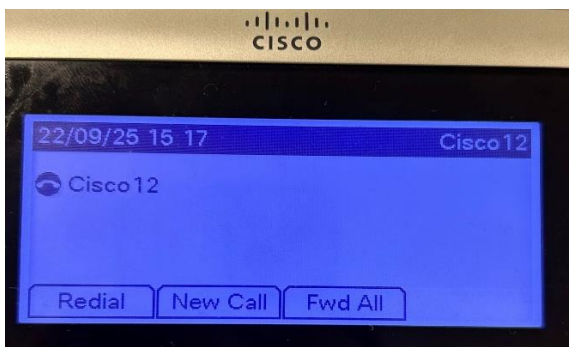
```
service in.tftpd restart
```

Une fois le serveur TFTP opérationnel, il faut configurer le téléphone CISCO pour qu'il charge sa configuration depuis le serveur TFTP :



- Adresse IP du serveur TFTP : **10.16.20.96**
- Adresse IP du téléphone : **10.17.1.32**

Cette configuration permet au téléphone CISCO de s'enregistrer automatiquement sur le serveur Asterisk avec les paramètres SIP du poste 12, conformément à la méthodologie professionnelle d'intégration des terminaux IP.



Voici un test à l'aide de la fonction ping. Nous constatons que notre serveur TFTP arrive bien à communiquer avec le téléphone Cisco12 :

```
* Starting tftpd ...
rt-mu:/var/tftpboot# ping 10.17.1.32
PING 10.17.1.32 (10.17.1.32): 56 data bytes
64 bytes from 10.17.1.32: seq=0 ttl=64 time=0.707 ms
64 bytes from 10.17.1.32: seq=1 ttl=64 time=0.592 ms
64 bytes from 10.17.1.32: seq=2 ttl=64 time=0.617 ms
64 bytes from 10.17.1.32: seq=3 ttl=64 time=0.589 ms
```

Illustration des deux méthodes d'enregistrement des terminaux

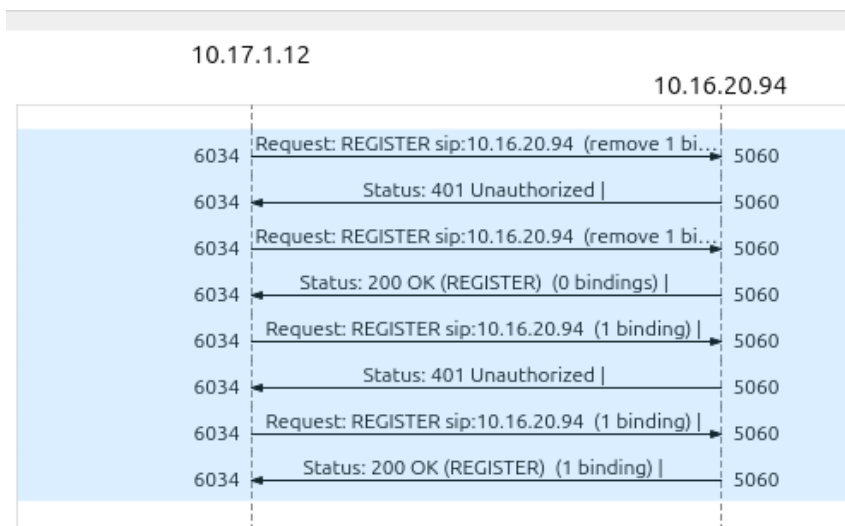
Pour cette étape, nous présentons les deux méthodes d'enregistrement des téléphones IP sur le serveur Asterisk : **enregistrement direct via SIP pour le Fanvil12** et **enregistrement via serveur TFTP pour le Cisco12**.

Méthode 1 – Enregistrement direct SIP (Fanvil12) :

- Lancer **Wireshark** sur la machine hôte et sélectionner l'interface **eth1**
- Capturer le trafic lors de l'enregistrement du Fanvil12 sur Asterisk
- Filtrer le protocole **SIP** afin de visualiser uniquement les messages relatifs à l'enregistrement

No.	Time	Source	Destination	Protocol	Length	Info
5988	11.468978956	10.17.1.12	10.16.20.94	SIP	568	Request: REGISTER sip:10.16.20.94 (remove 1 binding)
5989	11.470527525	10.16.20.94	10.17.1.12	SIP	561	Status: 401 Unauthorized
5990	11.475102955	10.17.1.12	10.16.20.94	SIP	836	Request: REGISTER sip:10.16.20.94 (remove 1 binding)
5991	11.476233535	10.16.20.94	10.17.1.12	SIP	458	Status: 200 OK (REGISTER) (0 bindings)
6291	17.678021406	10.17.1.12	10.16.20.94	SIP	571	Request: REGISTER sip:10.16.20.94 (1 binding)
6292	17.679070471	10.16.20.94	10.17.1.12	SIP	563	Status: 401 Unauthorized
6293	17.771425563	10.17.1.12	10.16.20.94	SIP	837	Request: REGISTER sip:10.16.20.94 (1 binding)
6294	17.772929705	10.16.20.94	10.17.1.12	SIP	513	Status: 200 OK (REGISTER) (1 binding)

- Générer un **diagramme de flux** pour illustrer l'échange entre le téléphone et le serveur



Maintenant, analysons les flux **SIP REGISTER** entre Fanvil ↔ Asterisk :

- **Première tentative** : le Fanvil12 envoie un message REGISTER sans authentification. En retour, le serveur Asterisk renvoie **401 Unauthorized**, demandant un mot de passe.
- **Deuxième tentative** : le Fanvil12 renvoie le REGISTER avec les identifiants corrects. La réponse du serveur est **200 OK**, l'enregistrement est validé.
- **Binding** : 1 binding créé, correspondant au paramètre max_contacts=1.

Cette méthode illustre l'enregistrement classique SIP utilisé pour la majorité des téléphones IP accessibles via interface Web.

Méthode 2 – Enregistrement via serveur TFTP (Cisco12) :

- Lancer **Wireshark** sur la machine hôte et sélectionner l'interface **eth1**
- Configurer un filtre de pré-capture sur l'adresse IP du serveur Asterisk et un filtre TFTP lors de la capture
- Vérifier le flux TFTP lors du téléchargement du fichier de configuration SEPXXXXXX.cnf.xml depuis le serveur TFTP vers le téléphone Cisco12

No.	Time	Source	Destination	Protocol	Length	Info
267	69.826307349	10.17.1.32	10.16.20.96	TFTP	73	Read Request, File: CTLSEP544A00369ECB.tlv, Transfer type: octet
268	69.826876530	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
276	70.484802202	10.17.1.32	10.16.20.96	TFTP	73	Read Request, File: ITLSEP544A00369ECB.tlv, Transfer type: octet
277	70.485336949	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
281	71.145171472	10.17.1.32	10.16.20.96	TFTP	62	Read Request, File: ITLFile.tlv, Transfer type: octet
282	71.145727591	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
286	72.018307603	10.17.1.32	10.16.20.96	TFTP	74	Read Request, File: SEP544A00369ECB.cnf.xml, Transfer type: octet
287	72.018905510	10.16.20.96	10.17.1.32	TFTP	64	Error Code, Code: Not defined, Message: Permission denied
310	77.518614874	10.17.1.32	10.16.20.96	TFTP	62	Read Request, File: /sp-sip.jar, Transfer type: octet
311	77.519212135	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
315	78.228303299	10.17.1.32	10.16.20.96	TFTP	64	Read Request, File: /g3-tones.xml, Transfer type: octet
316	78.228990463	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
322	79.237802352	10.17.1.32	10.16.20.96	TFTP	67	Read Request, File: DialTemplate.xml, Transfer type: octet
323	79.238381816	10.16.20.96	10.17.1.32	TFTP	64	Error Code, Code: Not defined, Message: Permission denied
325	79.333203791	10.17.1.32	10.16.20.96	TFTP	73	Read Request, File: CTLSEP544A00369ECB.tlv, Transfer type: octet
326	79.333916145	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
330	79.998245085	10.17.1.32	10.16.20.96	TFTP	73	Read Request, File: ITLSEP544A00369ECB.tlv, Transfer type: octet
331	79.998941487	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
336	80.665431617	10.17.1.32	10.16.20.96	TFTP	62	Read Request, File: ITLFile.tlv, Transfer type: octet
337	80.666191722	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
342	81.609948486	10.17.1.32	10.16.20.96	TFTP	74	Read Request, File: SEP544A00369ECB.cnf.xml, Transfer type: octet
343	81.610644199	10.16.20.96	10.17.1.32	TFTP	64	Error Code, Code: Not defined, Message: Permission denied
351	85.363536204	10.17.1.32	10.16.20.96	TFTP	62	Read Request, File: /sp-sip.jar, Transfer type: octet
352	85.364316607	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
356	86.031079754	10.17.1.32	10.16.20.96	TFTP	64	Read Request, File: /g3-tones.xml, Transfer type: octet
357	86.031769011	10.16.20.96	10.17.1.32	TFTP	61	Error Code, Code: File not found, Message: File not found
365	86.942764789	10.17.1.32	10.16.20.96	TFTP	67	Read Request, File: DialTemplate.xml, Transfer type: octet
366	86.943385209	10.16.20.96	10.17.1.32	TFTP	64	Error Code, Code: Not defined, Message: Permission denied

Maintenant, analysons les flux TFTP entre Cisco12 ↔ serveur TFTP :

Les fichiers de configuration doivent disposer des **droits d'accès complets** pour être accessibles par le téléphone. Plusieurs erreurs ont été constatées lors des premières tentatives, notamment l'apparition du message "**Registration in Progress**" sur le téléphone. Après vérification, le problème provenait uniquement d'une **mauvaise configuration des adresses IP** dans le fichier CNF. Après correction, le téléphone a pu récupérer le fichier via TFTP et s'enregistrer correctement sur le serveur Asterisk.

Nous devons pour cela rajouter tous les droits pour ces fichiers :

```
rt-mu:/var/tftpboot# ls -l
total 24
-rw-r--r-- 1 root root 2647 Mar 31 11:21 Archive.zip
-rwxrwxrwx 1 root root 214 Mar 12 2025 DialTemplate.xml
-rwxrwxrwx 1 root root 6648 Sep 17 11:55 SEP544A00369ECB.cnf.xml
-rw----- 1 root root 6645 Mar 12 2025 SEP_MAC-CISCO_.cnf.xml
```

Cette méthode illustre l'enregistrement automatisé utilisé pour les téléphones CISCO professionnels, qui ne permettent pas de modifier directement la configuration via l'interface Web.

Mise en place des boîtes vocales pour les deux comptes

Le déploiement de la **messagerie vocale** sur le serveur Asterisk se fait en configurant principalement deux fichiers :

- **voicemail.conf**
- **extensions.conf**

Créer le fichier **voicemail.conf** :

```
GNU nano 5.9                                voicemail.conf
[default]
1201 => 1234, Fanvil12
1202 => 5678, Cisco12
```

Modifier le fichier **extensions.conf** :

```
GNU nano 5.9                                extensions.conf
[[internal]
exten => 1201,1,Dial(PJSIP/Fanvil12,12)
exten=>1201,n,VoiceMail(${EXTEN}@default)
; Pour accéder aux boîtes vocales
exten=>888,1,VoiceMailMain($CALLERID(num}@default)

exten => 1202,1,Dial(PJSIP/Cisco12,12)
exten=>1202,n,VoiceMail(${EXTEN}@default)
; Pour accéder aux boîtes vocales
exten=>888,1,VoiceMailMain($CALLERID(num}@default)
```

Pour comprendre le fichier voici un exemple pour le compte Fanvil12 :

Numéro d'extension pour la messagerie	Mot de passe de la boîte vocale	Nom du compte	Durée avant bascule vers la messagerie
1201	1234	Fanvil12 (tel que défini dans pjsip.conf)	12 secondes (indiquée dans l'application Dial())

Règles générales pour la configuration des boîtes vocales :

- Le **numéro de la boîte vocale** correspond au **numéro de téléphone interne**.
- Le **contexte** utilisé est default.
- Si l'appelé ne décroche pas après **12 secondes**, l'appel bascule automatiquement sur la boîte vocale.
- Chaque boîte vocale dispose d'un **mot de passe unique** pour l'accès (exemple : 4567).
- Chaque utilisateur a un **numéro unique pour consulter sa messagerie**.

Cette configuration permet à chaque terminal de recevoir des messages en son absence et garantit une gestion individuelle et sécurisée de la messagerie vocale pour tous les comptes SIP.

Réalisation d'un échange voix entre les deux terminaux

Pour valider le fonctionnement du serveur Asterisk et des terminaux SIP, un **appel test** a été réalisé entre les deux postes, avec Wireshark activé sur l'interface **eth1** pour capturer le trafic.

Voici une description de l'échange :

- **Poste appelant** : 10.17.1.12 (Fanvil12)
- **Serveur SIP / Asterisk** : 10.16.20.94
- **Poste appelé** : 10.17.1.32 (Cisco12)

No.	Time	Source	Destination	Protocol	Length	Info
64	7.844141436	10.17.1.12	10.16.20.94	SIP/SDP	1066	Request: INVITE sip:1202@10.16.20.94;user=phone
65	7.845105146	10.16.20.94	10.17.1.12	SIP	570	Status: 401 Unauthorized
66	7.859111756	10.17.1.12	10.16.20.94	SIP	492	Request: ACK sip:1202@10.16.20.94;user=phone
67	7.853012915	10.17.1.12	10.16.20.94	SIP/SDP	1346	Request: INVITE sip:1202@10.16.20.94;user=phone
68	7.854102455	10.16.20.94	10.17.1.12	SIP	384	Status: 100 Trying
69	7.858127029	10.16.20.94	10.17.1.32	SIP/SDP	952	Request: INVITE sip:Cisco12@10.17.1.32:5060;transport=udp
70	7.865771847	10.17.1.32	10.16.20.94	SIP	1013	Status: 180 Trying
72	7.975891545	10.17.1.32	10.16.20.94	SIP	1107	Status: 180 Ringing
73	7.977228151	10.16.20.94	10.17.1.12	SIP	566	Status: 180 Ringing
163	19.865284945	10.16.20.94	10.17.1.32	SIP	460	Request: CANCEL sip:Cisco12@10.17.1.32:5060;transport=udp
164	19.865735127	10.16.20.94	10.17.1.12	SIP/SDP	876	Status: 200 OK (INVITE)
165	19.870607171	10.17.1.32	10.16.20.94	SIP	533	Status: 200 OK (CANCEL)
166	19.873613285	10.17.1.32	10.16.20.94	SIP	870	Status: 487 Request Cancelled
167	19.873968363	10.16.20.94	10.17.1.32	SIP	469	Request: ACK sip:Cisco12@10.17.1.32:5060;transport=udp
168	19.931900160	10.17.1.12	10.16.20.94	SIP	479	Request: ACK sip:10.16.20.94:5060
1174	32.670868706	10.17.1.12	10.16.20.94	SIP	489	Request: BYE sip:10.16.20.94:5060
1175	32.671372739	10.16.20.94	10.17.1.12	SIP	416	Status: 200 OK (BYE)

Établir un diagramme de flux de l'échange :



- Le poste Fanvil12 initie un appel vers Cisco12.
- L'appel est authentifié via SIP :
 - o Première tentative : **401 Unauthorized**
 - o Seconde tentative : nouvel **INVITE** avec identifiants corrects.
- Le poste appelé sonne (**180 Ringing**) mais ne décroche pas.
- L'appel est redirigé vers la **messagerie vocale**.
- Le message vocal déposé est : "Je souhaite te parler, rappelle-moi, merci."
- La session se termine avec **BYE / 200 OK**.

Cette analyse permet de démontrer que l'échange voix et le dépôt du message vocal sont correctement réalisés et fonctionnels sur l'infrastructure Asterisk mise en place.

Étape 3 - Accéder à l'administration d'Asterisk via son serveur HTTP

Cette étape consiste à mettre en place un serveur HTTP intégré à Asterisk pour permettre l'administration via une interface web simple, **sans utiliser de solutions tierces** telles que AsteriskNOW, FreePBX ou 3CX.

Vous devez utiliser le lien disponible http://www.asteriskdocs.org/en/2nd_Edition/asterisk-book-html-chunk/I_sect111_tt1345.html pour mettre en place le serveur intégré à Asterisk.

Activation du serveur HTTP intégré

Vérifier l'état du serveur HTTP Asterisk via CLI :

```
Connected to Asterisk 18.2.2 currently running on rt-mv (pid = 5913)
rt-mv*CLI> http show status
```

Nous avons en sortie :

```
HTTP Server Status:
Server: Asterisk
Server Enabled and Bound to 0.0.0.0:8088
Enabled URI's:
/httpstatus => Asterisk HTTP General Status
/manager => HTML Manager Event Interface w/Digest authentication
/ari/... => Asterisk RESTful API
/static/... => Asterisk HTTP Static Delivery
```

Maintenant, lancer la configuration du serveur HTTP via le fichier **http.conf** :

```
nano /etc/asterisk/http.conf
```

- Définir le port (8088 par défaut) et l'interface réseau.
- Indiquer le dossier contenant les fichiers HTML pour l'interface web.

```
GNU nano 5.9 /etc/asterisk/http.conf
[general]
servername=Asterisk
enabled=yes
bindaddr=0.0.0.0
bindport=8088
enable_static=yes
enable_status=yes
redirect = /var/lib/asterisk/static-http/mantest.html
```

Voici le dossier contenant nos fichiers HTML :

```
rt-mv:/var/lib/asterisk/static-http# ls
ajamdemo.html  astman.css    astman.js     mantest.html  prototype.js
```

Se rendre sur la page web ajamdemo.html pour tester le bon fonctionnement :

<http://10.16.20.94:8088/static/ajamdemo.html>

Développement d'une interface WEB simple

Créer un fichier HTML d'administration **admin.html** :

```
cd /var/lib/asterisk/static-http
nano admin.html
```

```
GNU nano 5.9                                admin.html
<!DOCTYPE html>
<html>
<head>
  <title>Mini Administration Asterisk</title>
</head>
<body>
<h1>Administration Asterisk (AJAM)</h1>

<button onclick="showPeers()">Voir les SIP peers</button>
<button onclick="showChannels()">Voir les canaux actifs</button>

<pre id="output" style="border:1px solid #000; padding:10px;"></pre>

<script>
function showPeers() {
  fetch('/arawman?username=asterisk_http&secret=goosey&action=Command&command=showPeers')
    .then(resp => resp.text())
    .then(data => document.getElementById('output').textContent = data);
}

function showChannels() {
  fetch('/arawman?username=asterisk_http&secret=goosey&action=Command&command=showChannels')
    .then(resp => resp.text())
    .then(data => document.getElementById('output').textContent = data);
}
</script>

</body>
</html>
```

Nous avons des champs pour nom d'utilisateur et mot de passe, ainsi que des boutons pour exécuter certaines actions : **voir les canaux actifs** et **voir les paires SIP**.

Définir un utilisateur avec accès HTTP dans **manager.conf** :

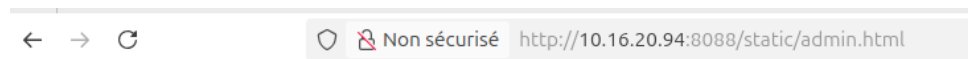
```
cd /etc/asterisk/manager.conf
```

```
GNU nano 5.9 manager.conf
[general]
enabled = yes
webenabled = yes

[asterisk_http] ; utilisateur pour l'interface web
secret = progr00
read = system,call,log,verbose,command,agent,user,config
write = system,call,log,verbose,command,agent,user,config
```

- Nom d'utilisateur : **asterisk_http**
- Mot de passe : **progr00**

Enfin, tester l'accès à la page : <http://10.16.20.94:8088/static/admin.html>



Administration Asterisk (AJAM)

[Voir les SIP peers](#) [Voir les canaux actifs](#)

```
Response: Success
Message: Command output follows
Output:
Output: Endpoint: <Endpoint/CID.....> <State.....> <Channels.>
Output:   I/OAuth: <AuthId/UserName.....>
Output:   Aor: <Aor.....> <MaxContact>
Output:   Contact: <Aor/ContactUri.....> <Hash.....> <Status> <RTT(ms)..>
Output:   Transport: <TransportId.....> <Type> <cos> <tos> <BindAddress.....>
Output:   Identify: <Identify/Endpoint.....>
Output:   Match: <criteria.....>
Output:   Channel: <ChannelId.....> <State.....> <Time.....>
Output:   Exten: <DialedExten.....> CLCID: <ConnectedLineCID.....>
Output: =====
Output: Endpoint: Cisco12                               Not in use    0 of inf
Output:   InAuth: Cisco12-auth/Cisco12
Output:   Aor: Cisco12                                     1
Output:   Contact: Cisco12/sip:Cisco12@10.17.1.32:5060;transp 644ab9f900 NonQual nan
Output:   Transport: transport-udp                          udp          0          0 0.0.0.0:5060
Output: Endpoint: Fanvil12                               Not in use    0 of inf
Output:   InAuth: Fanvil12-auth/Fanvil12
Output:   Aor: Fanvil12                                     5
Output:   Contact: Fanvil12/sip:Fanvil12@10.17.1.12:5306    4f94bcbad4 NonQual nan
Output:   Transport: transport-udp                          udp          0          0 0.0.0.0:5060
Output:
Output: Objects found: 2
Output:
```

Administration Asterisk (AJAM)

[Voir les SIP peers](#)[Voir les canaux actifs](#)

```
Response: Success
Message: Command output follows
Output: Channel                Location                State  Application(Data)
Output: PJSIP/Fanvil12-000000  (None)                Up     AppDial((Outgoing Line))
Output: PJSIP/Cisco12-000000  1201@internal:1       Up     Dial(PJSIP/Fanvil12,12)
Output: 2 active channels
Output: 1 active call
Output: 1 call processed
```

Une fois connecté, l'utilisateur peut consulter l'état du serveur, visualiser les appels en cours et gérer certains paramètres via une interface web simple, directement intégrée à Asterisk.

Cette approche permet une **administration web légère**, respectant la consigne de ne pas utiliser de solutions prépackagées, tout en restant fonctionnelle et sécurisée.

Étape 4 - Déployer un serveur WEB sur une machine virtuelle avec la distribution Alpine

L'objectif de cette étape est de déployer un serveur web sur une **machine virtuelle avec la distribution Alpine Linux** et de tester son bon fonctionnement avec une page d'accueil simple.

Il faudra créer une machine virtuelle avec la distribution Alpine de cette façon :

- Login : **root**
- Mot de passe : **progrtr00**
- Carte réseau : **bridge > eth1**
- Adresse IP : **10.16.20.194**

Tout d'abord, installer les outils nécessaires :

```
apk add nano
apk add openssh
service sshd start
```

Installation et configuration du serveur WEB

Exporter les variables proxy pour Alpine :

```
export http_proxy=http://cache-etu.univ-artois.fr:3128
export https_proxy=http://cache-etu.univ-artois.fr:3128
```

Mettre à jour les paquets et installer Apache2 :

```
apk update
apk add apache2
```

Configurer Apache :

- Fichier principal de configuration : `/etc/apache2/httpd.conf`

Créer une page d'accueil test **index.html** :

```
nano /var/index.htmlwww/localhost/htdocs/index.html
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Page d'accueil Apache</title>
</head>
<body>
  <h1>Mon serveur web Apache sur Alpine fonctionne !</h1>
  <p>Ceci est un test.</p>
```

```
</body>
</html>
```

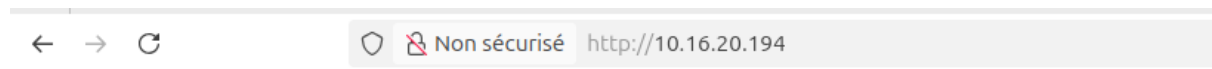
Assurer les droits sur les fichiers :

```
chown -R apache:apache /var/www/localhost/htdocs/
chmod -R 755 /var/www/localhost/htdocs/
```

Démarrer le serveur Apache et redémarrer si nécessaire :

```
httpd
httpd -k restart
```

Pour vérifier le bon fonctionnement, accéder à la page via un navigateur : <http://10.16.20.194>



Mon serveur web Apache sur Alpine fonctionne !

Ceci est un test.

Nous pouvons bien consulter le message : **"Mon serveur web Apache sur Alpine fonctionne !"**

Cette étape valide l'installation et le bon fonctionnement du serveur web, prêt pour l'intégration de la future interface de communication Asterisk.

Étape 5 - Intégrer une page WEB sur le serveur précédent pour réaliser un échange voix à l'aide de la librairie sip.js

Qu'est-ce que WebRTC ?

WebRTC (*Web Real-Time Communication*) est un ensemble de technologies open-source permettant d'établir des communications **audios, vidéos et données en temps réel directement depuis un navigateur web**, sans nécessiter de plugin ou de logiciel tiers.

WebRTC repose sur des **API JavaScript standardisées**, intégrées nativement dans les navigateurs modernes (Chrome, Firefox, Edge). Il permet la capture du microphone et de la caméra, l'établissement de connexions pair-à-pair sécurisées ou encore la transmission de flux multimédias en temps réel.

WebRTC s'appuie sur plusieurs protocoles clés :

- **ICE (Interactive Connectivity Establishment)** : découverte du chemin réseau optimal
- **STUN / TURN** : traversée des NAT et pare-feu
- **DTLS-SRTP** : chiffrement des flux audio/vidéo
- **RTP / SRTP** : transport des flux temps réel

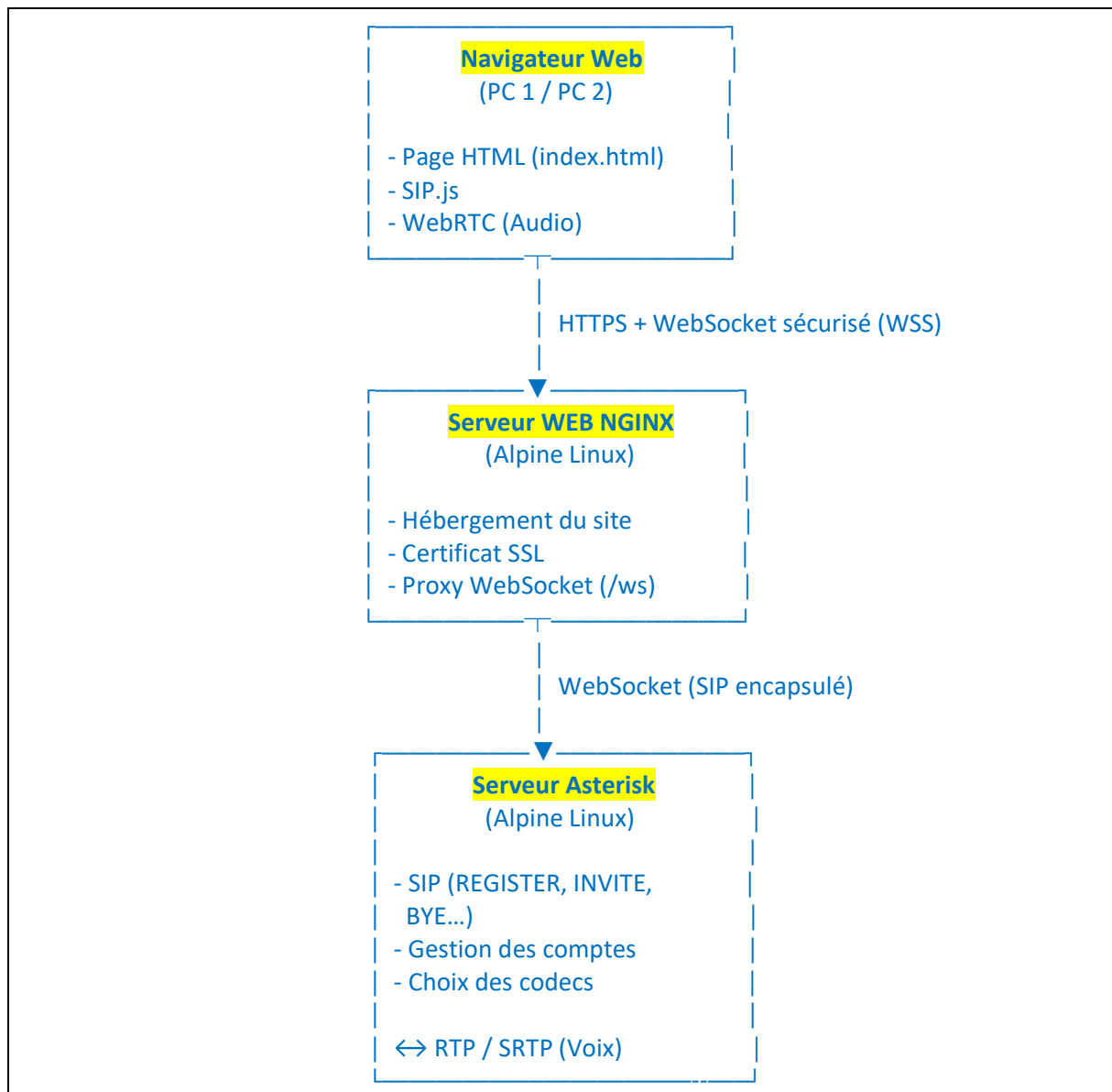
Dans notre projet, WebRTC est utilisé **uniquement pour la voix**, et s'appuie sur le serveur **Asterisk** pour la signalisation SIP.

Intégration avec SIP :

Afin de relier WebRTC au monde SIP, nous utilisons la librairie **SIP.js**, qui implémente le protocole SIP en JavaScript, utilise **WebSocket (WSS)** pour communiquer avec Asterisk, mais aussi elle permet l'enregistrement SIP et l'établissement d'appels depuis un navigateur.

Principe de mise en place pour la page WEB avec SIP.js

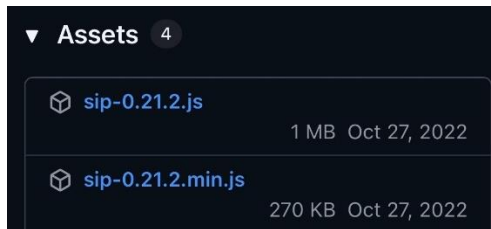
L'architecture mise en place est la suivante :



Le navigateur utilise WebRTC pour l'audio et SIP.js pour la signalisation. La signalisation SIP est transportée via WebSocket sécurisé vers le serveur NGINX, qui agit comme proxy. NGINX redirige ensuite les échanges vers Asterisk, lequel gère l'appel et le flux audio via RTP.

Configuration du serveur NGINX (Web + WebSocket)

Au préalable, avant de créer la page web il faudra **installer les fichiers sip.js**, que nous pouvons retrouver sur internet. Les fichiers JavaScript sont copiés dans le dossier **/www** via la commande **scp**.
(Prendre la version la plus récente)



Nous allons ensuite renommer ces fichiers de cette manière :

- sip-0.21.2.js → sip.js
- sip-0.21.2.min.js → sip.min.js

⚠ Important : les noms doivent correspondre exactement à ceux utilisés dans index.html.

Ensuite, nous allons configurer les fichiers nécessaires au bon fonctionnement du serveur web NGINX. Nous allons dans un premier temps configurer un fichier **default.conf** :

```
rt-mv:/etc/nginx/http.d# cat /etc/nginx/http.d/default.conf

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;

    root /www;
    index index.html;

    # Proxy WebSocket vers Asterisk distant
    location /ws {
        proxy_pass http://10.16.20.94:8088/ws; → Adresse du serveur Asterisk
        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;

        proxy_read_timeout 3600;
    }
}

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}
```

Ce fichier permet :

- l'hébergement du site web en HTTPS
- le proxy WebSocket sécurisé vers Asterisk
- la redirection HTTP → HTTPS

WebRTC impose l'utilisation du **HTTPS et du WSS**. Nous avons ainsi créé un dossier SSL :

```
sudo mkdir -p /etc/nginx/ssl
```

Puis, nous avons généré un **certificat auto-signé en 2048 bits** (pas en dessous, sinon ça ne fonctionnera pas).

Les fichiers générés sont :

- `/etc/nginx/ssl/server.crt`
- `/etc/nginx/ssl/server.key`

Concernant la création de la page web, une page **index.html** est créée dans le dossier **/www** du serveur WEB. Cela donne :

```
/www
├─ index.html
├─ sip.js
└─ sip.min.js
```

Le fichier **index.html** contient :

- l'interface graphique (HTML / CSS)
- la logique JavaScript WebRTC
- l'utilisation de la librairie **SIP.js** pour communiquer avec Asterisk

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Web SIP — Asterisk WebRTC Multi-PC</title>

  <!-- SIP.js chargé localement -->
  <script src="sip.min.js"></script>

  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh; padding: 20px; display: flex; justify-content: center; align-items: center; }
    .container { background: white; border-radius: 16px; box-shadow: 0 20px 60px rgba(0,0,0,0.3);
      max-width: 500px; width: 100%; padding: 30px; }
    h2 { color: #333; margin-bottom: 25px; text-align: center; font-size: 24px; }
    .section { background: #f8f9fa; border-radius: 12px; padding: 20px; margin-bottom: 20px;
      border: 2px solid #e9ecef; }
    .section-title { font-weight: 600; color: #495057; margin-bottom: 15px; font-size: 16px; text-
      transform: uppercase; letter-spacing: 0.5px; }
```

```

label { display: block; margin-top: 12px; font-weight: 500; color: #495057; font-size: 14px;
margin-bottom: 5px; }
input { width: 100%; padding: 10px 12px; border: 2px solid #dee2e6; border-radius: 8px; font-
size: 14px; transition: border-color 0.3s; }
input:focus { outline: none; border-color: #667eea; }
input:disabled { background: #e9ecef; cursor: not-allowed; }
.btn-group { display: flex; gap: 10px; margin-top: 15px; }
button { flex: 1; padding: 12px 20px; border: none; border-radius: 8px; font-size: 14px; font-
weight: 600; cursor: pointer; transition: all 0.3s; text-transform: uppercase; letter-spacing: 0.5px; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
.btn-primary { background: #667eea; color: white; }
.btn-danger { background: #e74c3c; color: white; }
.btn-success { background: #27ae60; color: white; }
.btn-warning { background: #f39c12; color: white; }
.status { padding: 12px; border-radius: 8px; margin-bottom: 20px; font-weight: 500; text-align:
center; font-size: 14px; }
.status-disconnected { background: #fee; color: #c0392b; border: 2px solid #e74c3c; }
.status-connected { background: #efd; color: #229954; border: 2px solid #27ae60; }
.status-calling { background: #fef3cd; color: #856404; border: 2px solid #ffc107; }
#log { margin-top: 20px; white-space: pre-wrap; background: #2c3e50; color: #ecf0f1; padding:
15px; border-radius: 8px; height: 300px; overflow-y: auto; font-family: 'Courier New', monospace;
font-size: 11px; line-height: 1.4; }
.audio-section { background: #f8f9fa; border-radius: 12px; padding: 20px; margin-top: 20px;
text-align: center; border: 2px solid #e9ecef; }
audio { width: 100%; margin-top: 10px; }
</style>
</head>
<body>
<div class="container">
<h2><img alt="WebRTC icon" data-bbox="175 550 195 565"/> WebRTC SIP Multi-PC</h2>

<div id="status" class="status status-disconnected"><img alt="Red status icon" data-bbox="530 585 550 600"/> Déconnecté</div>

<div class="section">
<div class="section-title"><img alt="Server icon" data-bbox="340 635 360 650"/> Configuration Serveur</div>
<label>Serveur SIP</label>
<input id="sipServer" value="10.16.20.94" />
<label>WebSocket URL</label>
<input id="wsUrl" value="wss://10.16.21.6/ws" />
</div>

<div class="section">
<div class="section-title"><img alt="User icon" data-bbox="340 765 360 780"/> Authentification</div>
<label>Extension</label>
<input id="username" value="1000" />
<label>Mot de passe</label>
<input id="password" value="1000pass" type="password" />
<div class="btn-group">
<button id="btnRegister" class="btn-primary">S'enregistrer</button>
<button id="btnUnregister" class="btn-danger" disabled>Déconnecter</button>
</div>

```

```

</div>

<div class="section">
  <div class="section-title">📞 Appel / Décrocher</div>
  <label>Numéro à appeler</label>
  <input id="target" value="1001" />
  <div class="btn-group">
    <button id="btnCall" class="btn-success" disabled>Appeler</button>
    <button id="btnAnswer" class="btn-warning" disabled>Décrocher</button>
    <button id="btnHangup" class="btn-danger" disabled>Raccrocher</button>
  </div>
</div>

<div class="audio-section">
  <div class="section-title">🔊 Audio distant</div>
  <audio id="remoteAudio" autoplay controls></audio>
</div>

<div id="log"></div>
</div>

<script>
(async () => {
  const logEl = (msg,isError=false)=>{
    const el=document.getElementById('log');
    const prefix=isError?' ❌ ❌ ❌ ':' ';
    const timestamp=new Date().toLocaleTimeString();
    el.textContent=`${prefix}[${timestamp}] ${msg}\n`+el.textContent;
  };

  const updateStatus=(status,text)=>{
    const statusEl=document.getElementById('status');
    statusEl.className=`status status-${status}`;
    statusEl.textContent=text;
  };

  let userAgent=null, registerer=null, session=null, localStream=null, pendingInvite=null;

  const btnRegister=document.getElementById('btnRegister');
  const btnUnregister=document.getElementById('btnUnregister');
  const btnCall=document.getElementById('btnCall');
  const btnAnswer=document.getElementById('btnAnswer');
  const btnHangup=document.getElementById('btnHangup');
  const remoteAudio=document.getElementById('remoteAudio');

  async function ensureMedia(){
    if(!localStream){
      try{
        localStream=await navigator.mediaDevices.getUserMedia({audio:true,video:false});
        logEl("✅ Microphone activé");
      }catch(e){

```

```

        logEl("✖ Erreur micro: "+e.message,true);
        alert("Micro non accessible"); throw e;
    }
}
return localStream;
}

function buildUserAgent(username,password){
    const sipServer=document.getElementById('sipServer').value.trim();
    const wsUrl=document.getElementById('wsUrl').value.trim();
    const uri=SIP.UserAgent.makeURI(`sip:${username}@${sipServer}`);
    return new SIP.UserAgent({
        uri,
        transportOptions:{server: wsUrl, wsProtocols:["sip"]},
        authorizationUsername: username,
        authorizationPassword: password,
        delegate:{
            onInvite: async(inv)=>{
                logEl("📞 Appel entrant...");
                pendingInvite=inv;
                btnAnswer.disabled=false;
            }
        }
    });
}

function setupSession(s){
    session=s;
    btnHangup.disabled=false;
    btnCall.disabled=true;
    btnAnswer.disabled=true;
    updateStatus('calling','📞 En communication');

    const pc=s.sessionDescriptionHandler?.peerConnection;
    if(pc){
        pc.ontrack=(ev)=>{ if(ev.streams && ev.streams[0]) remoteAudio.srcObject=ev.streams[0]; };
        pc.oniceconnectionstatechange=()=>{ logEl("ICE state: "+pc.iceConnectionState); };
    }

    s.stateChange.addListener((state)=>{
        if(state===SIP.SessionState.Terminated) cleanupSession();
    });
}

function cleanupSession(){
    logEl("📞 Session terminée");
    session=null;
    pendingInvite=null;
    btnHangup.disabled=true;
    btnCall.disabled=false;
    btnAnswer.disabled=true;
}

```

```

remoteAudio.srcObject=null;
updateStatus('connected',' Connecté');
}

btnRegister.onclick=async()=>{
  const username=document.getElementById('username').value.trim();
  const password=document.getElementById('password').value;
  if(!username || !password){ alert("Remplir extension/mot de passe"); return; }
  await ensureMedia();
  if(userAgent){ logEl("  Déjà connecté",true); return; }

  updateStatus('calling',' Connexion...');
  userAgent=buildUserAgent(username,password);
  userAgent.start()
    .then(()=>{
      registerer=new SIP.Registerer(userAgent);
      registerer.register()
        .then(()=>{
          logEl("  Enregistré "+username);
          updateStatus('connected',' Connecté - '+username);
          btnUnregister.disabled=false;
          btnRegister.disabled=true;
          btnCall.disabled=false;
        })
        .catch(err=>{
          logEl("  Enregistrement fail: "+err.message,true);
          updateStatus('disconnected',' Échec');
          userAgent.stop(); userAgent=null;
        });
    })
    .catch(err=>{
      logEl("  UA start error: "+err.message,true);
      updateStatus('disconnected',' Erreur UA');
      userAgent=null;
    });
};

btnUnregister.onclick=async()=>{
  if(registerer) await registerer.unregister();
  if(userAgent) await userAgent.stop();
  userAgent=null; session=null; pendingInvite=null;
  updateStatus('disconnected',' Déconnecté');
  btnRegister.disabled=false;
  btnUnregister.disabled=true;
  btnCall.disabled=true;
  btnAnswer.disabled=true;
  btnHangup.disabled=true;
};

btnCall.onclick=async()=>{

```

```

const target=document.getElementById('target').value.trim();
const sipServer=document.getElementById('sipServer').value.trim();
if(!userAgent){ alert("Veuillez vous enregistrer"); return; }
await ensureMedia();

const targetURI=SIP.UserAgent.makeURI(`sip:${target}@${sipServer}`);
const inviter=new
SIP.Inviter(userAgent,targetURI,{sessionDescriptionHandlerOptions:{constraints:{audio:true,video:
false}}});
inviter.stateChange.addListener((state)=>{ logEl("📊 Inviter state: "+state); });
inviter.invite().then(()=>{ setupSession(inviter); }).catch(e=>{ logEl("❌ Appel fail:
"+e.message,true); });
};

btnAnswer.onclick=async()=>{
if(!pendingInvite){ logEl("⚠️ Aucun appel entrant",true); return; }
await ensureMedia();
try{
await
pendingInvite.accept({sessionDescriptionHandlerOptions:{constraints:{audio:true,video:false}}});
setupSession(pendingInvite);
}catch(e){ logEl("❌ Erreur accept: "+e.message,true); }
};

btnHangup.onclick=async()=>{ if(session) await session.bye(); };

logEl("🚀 Interface WebRTC initialisée");
})();
</script>
</body>
</html>

```


Configuration côté Asterisk (WebRTC)

Nous avons créé un fichier **pjsip.conf** pour les utilisateurs WebRTC. Point important, le **contexte doit être internal** et surtout **pas** from-internal.

Création de l'utilisateur 1000 :

```
; =====  
; Utilisateurs WebRTC  
; =====  
[1000]  
type=endpoint  
transport=transport-wss  
context=internal  
disallow=all  
allow=opus,ulaw,alaw  
webrtc=yes  
ice_support=yes  
rtp_symmetric=yes  
force_rport=yes  
rewrite_contact=yes  
auth=auth1000  
aors=1000  
media_encryption=dtls  
dtls_auto_generate_cert=yes  
  
[auth1000]  
type=auth  
auth_type=userpass  
password=1000pass  
username=1000  
  
[1000]  
type=aor  
max_contacts=1
```

Création de l'utilisateur 1001 :

```
; ----- Extension 1001  
[1001]  
type=endpoint  
transport=transport-wss  
context=internal  
disallow=all  
allow=opus,ulaw,alaw  
webrtc=yes  
ice_support=yes  
rtp_symmetric=yes  
force_rport=yes  
rewrite_contact=yes  
auth=auth1001  
aors=1001  
media_encryption=dtls  
dtls_auto_generate_cert=yes  
  
[auth1001]  
type=auth  
auth_type=userpass  
password=1001pass  
username=1001  
  
[1001]  
type=aor  
max_contacts=1
```

Ensuite, configurer le dialplan du fichier **extensions.conf** (déjà existant) pour 1000 et 1001 :

```
[internal]  
exten => 1201,1,Dial(PJSIP/Fanvil12,20)  
exten => 1202,1,Dial(PJSIP/Cisco12,20)  
  
; Appels WebRTC ↔ WebRTC  
exten => 1000,1,NoOp(Appel vers softphone 1000)  
same => n,Dial(PJSIP/1000,20)  
same => n,Hangup()  
  
exten => 1001,1,NoOp(Appel vers softphone 1001)  
same => n,Dial(PJSIP/1001,20)  
same => n,Hangup()
```

Après modification, nous devons redémarrer le serveur Asterisk :

```
asterisk -rx "pjsip reload"  
ou  
asterisk -rx "core restart now"
```

Enfin, vérifier les codecs disponibles par notre serveur :

```
asterisk -rx "core show codecs"
```

Nous constatons que beaucoup de codecs ne sont pas installés.

Installation du codec OPUS (recommandé pour WebRTC) :

```
apk add asterisk-opus
```

Chargement du module OPUS : `asterisk -rx "module load codec_opus_open_source.so"`

Puis, vérifier que le module est bien installé : `asterisk -rx "module show like opus"`

```
rt-mv:/etc/asterisk# asterisk -rx "module show like opus"
Module                               Description                               Use
Count  Status      Support Level
codec_opus_open_source.so            Opus Coder/Decoder                       0
    Running              unknown
res_format_attr_opus.so              Opus Format Attribute Module             1
    Running              core
2 modules loaded
```

⚠ Important : après chaque modification redémarrer le serveur Asterisk

Le codec OPUS est installé sur le serveur Asterisk car il s'agit du codec audio natif utilisé par WebRTC. Son installation est indispensable afin d'assurer la compatibilité entre les navigateurs web et Asterisk, et de garantir une transmission audio de qualité lors des appels SIP WebRTC.

Tests et validation de la solution WebRTC / SIP

1) Accès à l'interface Web SIP

Avant connexion (test avec l'utilisateur 1000) :

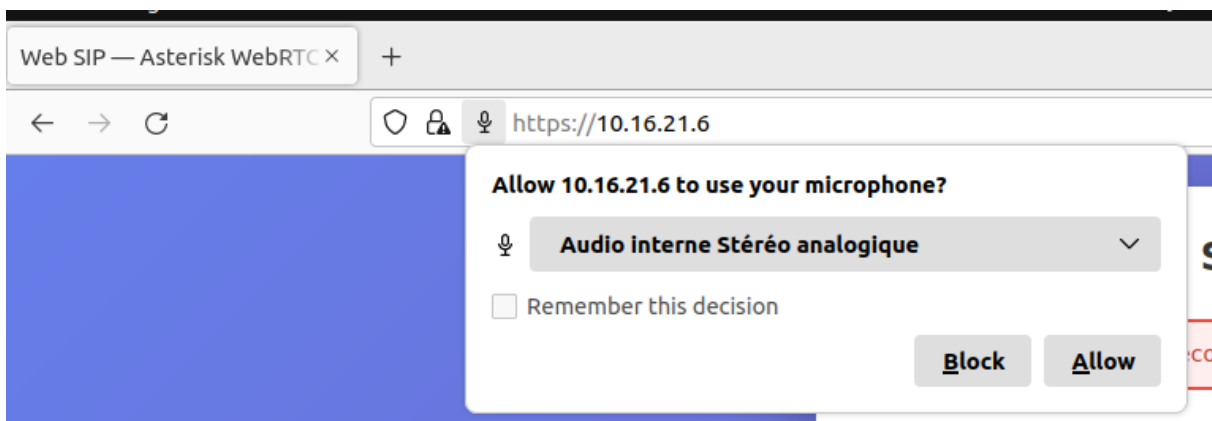
Vérifier que l'interface Web est accessible et fonctionnelle avant authentification.

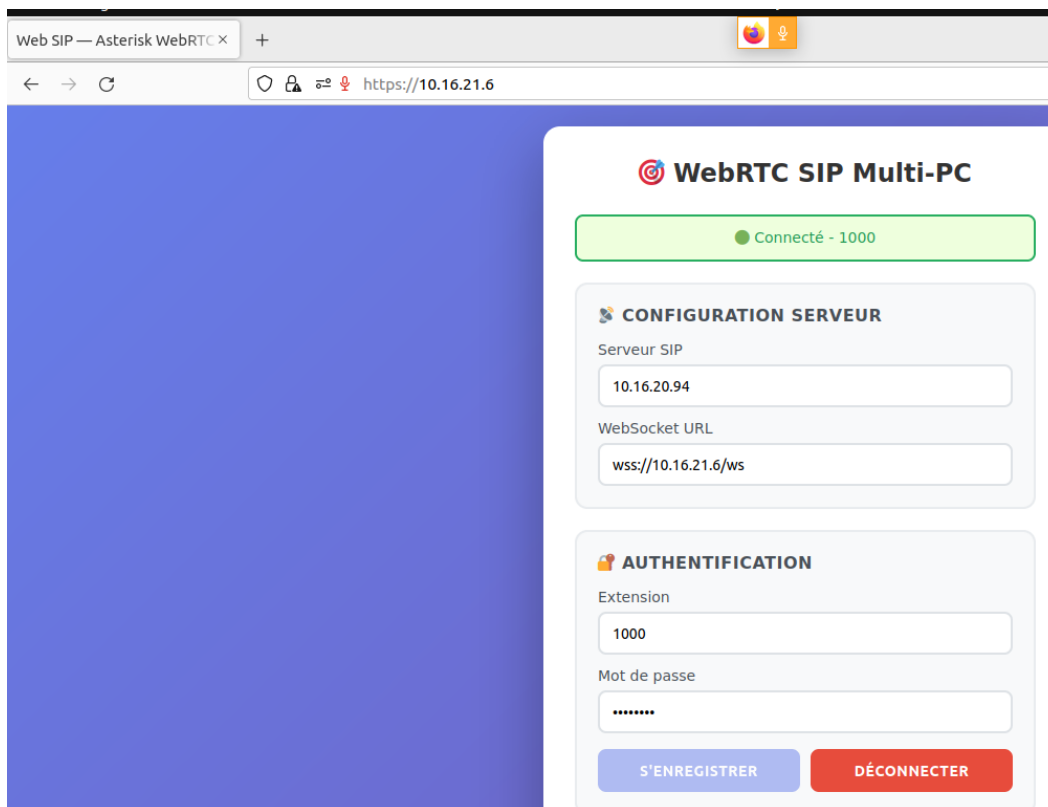


The screenshot shows the 'WebRTC SIP Multi-PC' interface. At the top, there's a status bar indicating 'Déconnecté'. Below this, the 'CONFIGURATION SERVEUR' section contains two input fields: 'Serveur SIP' with the value '10.16.20.94' and 'WebSocket URL' with the value 'wss://10.16.21.6/ws'. The 'AUTHENTIFICATION' section has two input fields: 'Extension' with the value '1000' and 'Mot de passe' with masked characters. At the bottom, there are two buttons: 'S'ENREGISTRER' (blue) and 'DÉCONNECTER' (red).

Après connexion (test avec l'utilisateur 1000) :

Vérifier l'enregistrement SIP via WebSocket (WSS).





2) Tests d'appels depuis l'interface WebRTC (côté serveur Asterisk)

Appel WebRTC (1000) → Fanvil 12 :

Tester un appel depuis un client WebRTC vers un téléphone SIP matériel.

```
=====
Connected to Asterisk 18.2.2 currently running on rt-mv (pid = 3247)
Core debug is still 3.
  WebSocket connection from '10.16.21.6:43246' for protocol 'sip' accepted using version '13'
  Added contact 'sip:eascu5tl@10.16.21.6:43246;transport=WS' to AOR '1000' with expiration of 600 seconds
  Endpoint 1000 is now Reachable
  Executing [1201@internal:1] NoOp("PJSIP/1000-00000000", "Appel vers Fanvil12") in new stack
  Executing [1201@internal:2] Dial("PJSIP/1000-00000000", "PJSIP/Fanvil12,20") in new stack
  Called PJSIP/Fanvil12
  PJSIP/Fanvil12-00000001 is ringing
```

Appel WebRTC (1000) → Cisco 12 :

Vérifier la compatibilité avec un autre type de terminal SIP.

```
Executing [1202@internal:1] Dial("PJSIP/1000-00000002", "PJSIP/Cisco12,20") in new stack
Called PJSIP/Cisco12
PJSIP/Cisco12-00000003 is ringing
```

Appel WebRTC (1000) → WebRTC (1001) :

Tester une communication voix entre deux navigateurs Web.

```
=====
Connected to Asterisk 18.2.2 currently running on rt-mv (pid = 3247)
Core debug is still 3.
- WebSocket connection from '10.16.21.6:43250' for protocol 'sip' accepted using version '13'
- Added contact 'sip:33f0abg5@10.16.21.6:43250;transport=WS' to AOR '1001' with expiration of 600 seconds
- Endpoint 1001 is now Reachable
- Executing [1000@internal:1] NoOp("PJSIP/1001-00000004", "Appel vers softphone 1000") in new stack
- Executing [1000@internal:2] Dial("PJSIP/1001-00000004", "PJSIP/1000,20") in new stack
- Called PJSIP/1000
- Everyone is busy/congested at this time (1:0/1/0)
- Executing [1000@internal:3] Hangup("PJSIP/1001-00000004", "") in new stack
- Spawn extension (internal, 1000, 3) exited non-zero on 'PJSIP/1001-00000004'
```

Étape 6 – Compléter l'interface précédente pour intégrer une messagerie instantanée

L'objectif de cette étape est d'enrichir l'interface Web existante en y intégrant une fonctionnalité de messagerie instantanée permettant l'échange de messages textuels entre deux postes clients, PC1 (extension 1000) et PC2 (extension 1001).

Choix technique : Alternative au protocole SIP

Initialement, l'utilisation du protocole **SIP MESSAGE** via Asterisk avait été envisagée. Cependant, au vu de la complexité de mise en œuvre du module PJSIP avec WebRTC et des instabilités rencontrées lors des phases précédentes (problématiques de TLS et de gestion des certificats), nous avons opté pour une solution plus robuste et légère : une messagerie basée sur une **architecture HTTP/JSON**.

Cette approche s'appuie sur le serveur Web déjà déployé pour gérer les requêtes via une API simple en PHP, utilisant un fichier JSON partagé pour le stockage des données.

Préparation de l'environnement PHP sur Alpine Linux

Pour supporter l'exécution de scripts côté serveur, nous avons installé PHP 8 ainsi que le serveur Web `lighttpd` sur la machine virtuelle dédiée (10.16.21.6) :

```
apk add php8 php8-cgi lighttpd
service lighttpd start
```

Afin que le serveur puisse interpréter les scripts PHP, nous avons modifié le fichier de configuration `/etc/lighttpd/lighttpd.conf` :

```
nano /etc/lighttpd/lighttpd.conf
```

➔ Nous avons modifié la section **server.modules** pour inclure les bibliothèques indispensables au projet :

```
server.modules = (
    "mod_access",
    "mod_accesslog",
    "mod_indexfile",
    "mod_fastcgi", # Module pour PHP
    "mod_cgi",
    "mod_dirlisting",
    "mod_staticfile",
    "mod_openssl" # Module pour le HTTPS
)
```

- **mod_fastcgi** : Pour la liaison avec l'interpréteur PHP.
- **mod_openssl** : Pour la sécurisation des échanges via SSL/TLS (HTTPS).

➔ Nous avons également défini l'interpréteur pour les fichiers .php :

```
# Association de l'extension .php à l'exécutable PHP-CGI
fastcgi.server = (
    ".php" => (
        "php-handler" => (
            "bin-path" => "/usr/bin/php-cgi8",
            "socket" => "/tmp/php.socket",
            "max-procs" => 1,
            "bin-environment" => (
                "PHP_FCGI_CHILDREN" => "1",
                "PHP_FCGI_MAX_REQUESTS" => "10000"
            ),
            "broken-scriptfilename" => "enable"
        )
    )
)
```

Une fois la configuration enregistrée, le service est redémarré pour appliquer les changements :

```
service lighttpd restart
```

Création de l'API de messagerie (chat.php)

Nous avons développé un script PHP situé dans **/www** pour gérer les flux de messages. Ce script fait office de "backend" simplifié.

Code source du script **chat.php** :

```
<?php
// chat.php - API simple de messagerie
header('Content-Type: application/json; charset=utf-8');

$file = __DIR__ . '/messages.json';

// Si le fichier n'existe pas, on le crée vide
if (!file_exists($file)) {
    file_put_contents($file, json_encode([]));
}

$messages = json_decode(file_get_contents($file), true);
if (!is_array($messages)) {
    $messages = [];
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // On attend du JSON dans le body
    $raw = file_get_contents('php://input');
    $data = json_decode($raw, true);
}
```

```

if (!is_array($data) || !isset($data['user']) || !isset($data['message'])) {
    echo json_encode(['status' => 'error', 'message' => 'invalid data']);
    exit;
}

$messages[] = [
    'user' => $data['user'],
    'message' => $data['message'],
    'time' => date('H:i:s'),
];

file_put_contents($file, json_encode($messages));
echo json_encode(['status' => 'ok']);
exit;
}

// Si ce n'est pas un POST (requête GET) -> on renvoie toute la liste
echo json_encode($messages);
exit;
?>

```

Fonctionnement du script :

- **En mode POST** : Il reçoit un message au format JSON (contenant l'utilisateur et le texte), lui ajoute un horodatage système, et l'écrit dans le fichier messages.json.
- **En mode GET** : Il renvoie l'ensemble de l'historique des messages sous forme de tableau JSON pour que les clients Web puissent l'afficher.

Mise en place du stockage des messages

Pour centraliser les échanges, nous utilisons un fichier JSON nommé **messages.json** situé à la racine du serveur Web. Il est crucial de définir les droits d'écriture corrects pour que le serveur puisse modifier le fichier :

```

cd /www
echo "[]" > messages.json
chmod 666 messages.json

```

Enfin, nous avons adapté notre page **index.html** pour intégrer une zone d'affichage des messages et un champ d'envoi utilisant JavaScript (fetch). La messagerie est désormais fonctionnelle et accessible de manière sécurisée à l'adresse suivante : <https://10.16.21.6/index.html>

Tests et validation de la messagerie instantanée :

Scénario 1 : Échange entre l'extension 1000 (Noa PC1) et 1001 (Indira PC2)

Une fois que PC1 (Noa) s'est bien connecté, il peut accéder à la messagerie instantanée. Ici, nous avons une discussion avec l'extension 1001.

The screenshot shows a web interface with two main sections. The top section, titled 'AUTHENTIFICATION', contains a form with 'Extension' (1000) and 'Mot de passe' (masked with dots). Below the form are two buttons: 'S'ENREGISTRER' (blue) and 'DÉCONNECTER' (red). The bottom section, titled 'MESSAGERIE INSTANTANÉE', features a dropdown menu for 'Destinataire' set to 'Extension 1001 (Indira)'. The chat area shows a conversation: a grey bubble from 1001 at 14:20 saying 'Saluuuut Noa ! Alors ça marche ?' and a blue bubble from 1000 at 14:21 saying 'Oui, je suis connecté'. At the bottom is a 'Message...' input field and an 'ENVOYER' button.

Les deux extensions peuvent communiquer ensemble et l'heure se met à jour avec celle du PC :

This screenshot shows the 'MESSAGERIE INSTANTANÉE' interface with the 'Destinataire' set to 'Extension 1001 (Indira)'. The chat history now includes three messages: a blue bubble from 1000 at 14:21 saying 'Oui, je suis connecté', a blue bubble from 1000 at 18:06 saying 'Salut Indidi', and a grey bubble from 1001 at 18:06 saying 'Bien reçu Noa -_-'. The 'Message...' input field and 'ENVOYER' button are at the bottom.

Scénario 2 : Échange entre l'extension 1000 (Noa PC1) et 1002 (Florentin PC3)

Nous avons ici l'extension 1002 qui écrit à 1000.

The screenshot displays a user interface for a messaging system. At the top, it says "MESSAGERIE INSTANTANÉE". Below this, there is a "Destinataire :" (Recipient) dropdown menu currently set to "Extension 1002 (Florentin)". The message history shows a single incoming message from "1002 • 15:05" with the text "Hello Noa le bg". At the bottom, there is a text input field labeled "Message..." and a blue button labeled "ENVOYER".

De la même manière que tout à l'heure, 1000 peut répondre à 1002 avec l'heure qui se met à jour :

This screenshot shows the same messaging interface after a second message. The "Destinataire" remains "Extension 1002 (Florentin)". The message history now contains two messages: the first is the incoming "1002 • 15:05" message "Hello Noa le bg", and the second is an outgoing message from "1000 • 18:11" with the text "Salut flo". The text input field and "ENVOYER" button are still present at the bottom.

Les problèmes rencontrés

La connexion entre les deux ordinateurs n’a jamais abouti. Après une série de tests et une analyse poussée, nous avons découvert que la source du problème venait de la version d’Asterisk utilisée, qui nous obligeait à passer par le module PJSIP plutôt que par SIP.

En théorie, PJSIP est censé fonctionner avec WebRTC, mais dans notre configuration, la combinaison des versions d’Asterisk et de PJSIP a créé des incompatibilités persistantes. Nous nous sommes heurtés à la complexité des mécanismes nécessaires – TLS, SRTP, ICE, la gestion des certificats et du NAT – sans parvenir à les faire fonctionner ensemble de manière stable.

Malgré plusieurs ajustements et séances de débogage, les deux postes sont restés muets. Faute de temps et de moyens pour creuser davantage, nous n’avons pas pu surmonter ces obstacles techniques, ce qui a finalement empêché la réalisation du projet.

Conclusion

Ce projet avait pour objectif de mettre en place une infrastructure complète de téléphonie IP reposant sur le serveur **Asterisk**, intégrant à la fois des **terminaux SIP matériels**, des **postes logiciels**, ainsi qu'une **interface Web basée sur WebRTC**. À travers les différentes étapes, nous avons progressivement construit et validé chaque brique du système.

Dans un premier temps, le déploiement d'Asterisk a permis de configurer des comptes SIP, d'assurer l'enregistrement des terminaux Fanvil et Cisco, de mettre en place un serveur TFTP pour le provisioning, ainsi que de configurer les boîtes vocales. L'analyse des flux réseau à l'aide de **Wireshark** a permis de comprendre en détail les mécanismes de signalisation SIP et de transport audio via RTP, confirmant le bon fonctionnement de la solution VoIP classique.

Dans un second temps, nous avons déployé un **serveur Web sous Alpine Linux**, configuré avec **NGINX**, afin d'héberger une interface WebRTC utilisant la librairie **sip.js**. Cette étape a permis de mettre en évidence les interactions entre les technologies Web (HTTP, HTTPS, WebSocket sécurisé) et le monde de la téléphonie IP. Les appels entre un client WebRTC et des terminaux SIP matériels ont pu être établis, démontrant la capacité d'Asterisk à jouer le rôle de passerelle entre WebRTC et SIP.

Cependant, malgré une configuration fonctionnelle sur une grande partie du projet, un problème majeur est apparu lors de la tentative de communication voix entre **deux navigateurs WebRTC**. La connexion entre les deux ordinateurs n'a jamais abouti. Après une série de tests et une analyse approfondie, nous avons identifié que la source du problème provenait principalement de la **version d'Asterisk utilisée**, qui imposait l'utilisation du module **PJSIP** plutôt que l'ancien module SIP.

Bien que PJSIP soit théoriquement compatible avec WebRTC, la combinaison spécifique des versions d'Asterisk et de PJSIP a engendré des **incompatibilités persistantes** dans notre environnement. Nous nous sommes heurtés à la complexité des mécanismes nécessaires au bon fonctionnement de WebRTC, notamment **TLS, SRTP, ICE, la gestion des certificats, ainsi que les problématiques liées au NAT**. Malgré plusieurs ajustements de configuration et différentes séances de débogage, aucun flux audio n'a pu être établi entre les deux postes WebRTC.

Faute de temps et de moyens supplémentaires pour approfondir davantage ces aspects avancés, il n'a pas été possible de lever ces blocages techniques. Néanmoins, ce projet a permis d'acquérir une compréhension approfondie des architectures VoIP modernes, des protocoles SIP et WebRTC, ainsi que des difficultés réelles rencontrées lors de leur intégration.

En conclusion, même si l'objectif final de communication WebRTC complète n'a pas pu être atteint, le projet reste **globalement abouti et formateur**, tant sur le plan technique que méthodologique. Il met en évidence les enjeux et la complexité des infrastructures de téléphonie IP actuelles et constitue une base solide pour de futures améliorations.